3강. React 개발환경 및 컴포넌트

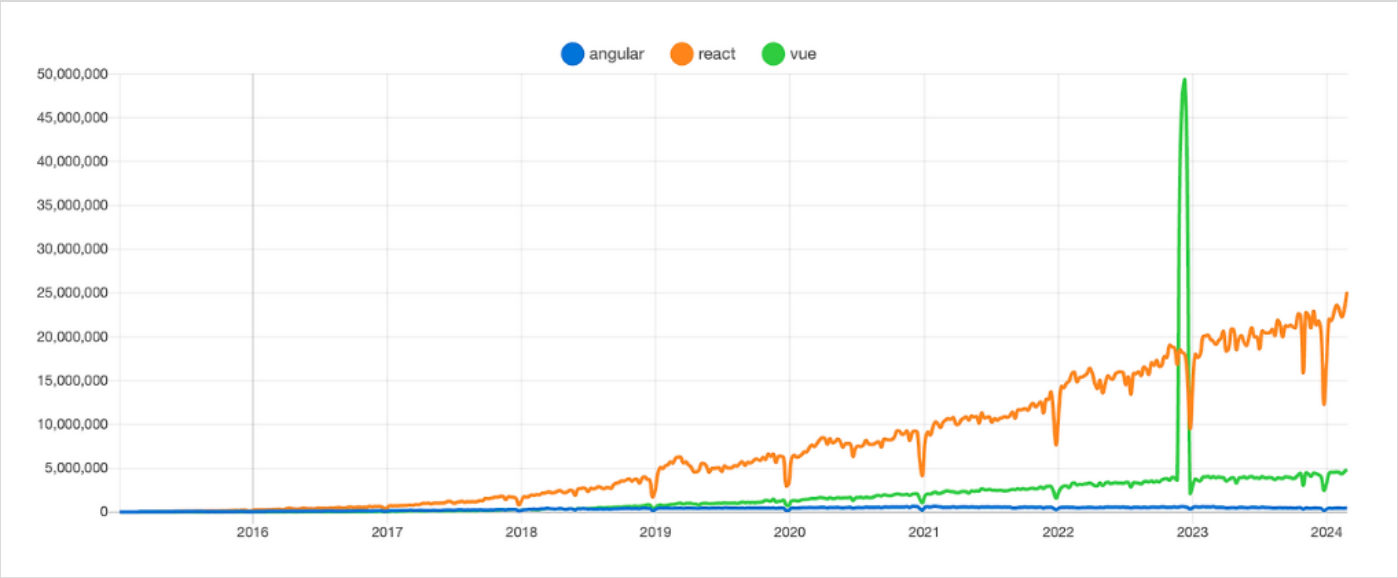
React

리엑트(React)란?

- **리엑트(React)란?**

  리엑트는 **사용자 인터페이스(UI, User Interface)를 구축하기 위한 자바스크립트 프레임워크**입니다.

  2013년 페이스북(현, 메타)에서 처음 공개되었으며, 현재는 가장 널리 사용하는 프런트엔드 라이브러리로 위치하고 있습니다.



리엑트(React)의 특징

- **SPA(Single Page Application)**

SPA란 한 개의 HTML 페이지로 동작하는 웹 애플리케이션이다.

즉 페이지 이동이 없고, 필요한 부분만 바뀜

- **기존 웹 방식**

사용자 클릭
↓
서버 요청
↓
새 HTML 전체 응답
↓
페이지 새로고침

☞ 대표 특징
- 화면 깜빡임
- 느린 전환
- 서버 의존적

리엑트(React)의 특징

- **SPA(Single Page Application)**
  - **SPA 방식**

```
사용자 클릭
↓
JavaScript 실행
↓
필요한 데이터만 요청 (API)
↓
화면 일부만 변경
```

☞ 대표 특징
- 빠른 화면 전환
- 부드러운 UX
- 클라이언트 중심

  - **예) 쇼핑몰**

[상품 목록] → 클릭 → [상품 상세]

- 페이지 이동 ❌
- URL만 변경 ⭕
- 내용만 교체 ⭕

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

1 . 컴포넌트 기반 구조

재사용 가능한 컴포넌트 단위로 나누어 개발함

```
components
  ├ Header
  ├ Menu
  └ Footer
```

```
function Hello() {
  return <h1>Hello React</h1>;
}
```

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

2. 가상돔(Virtual DOM) 사용 - 메모리에 존재하는 가짜 DOM

- DOM(Document Object Model) 트리

```
<html>
 ├ <body>
    ├ <h1>Hello</h1>
    └ <p>Text</p>
```

- 일반 JavaScript 방식

```
document.getElementById("title").innerText = "Hello React";
```

- DOM 변경 → 화면 다시 그리기(Reflow, Repaint)
- 특히 요소가 많으면 느려짐
- 성능 저하 발생

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

2. 가상돔(Virtual DOM) 사용

일반 웹은 **DOM 전체(브라우저에서)를 다시 렌더링**할 수 있지만 React는 **Virtual DOM**을 사용합니다.

☞ 동작 방식

```
[상태 변경]
      ↓
[새 Virtual DOM 생성]
      ↓
[이전 Virtual DOM과 비교]
      ↓
[바뀐 부분 찾기]
      ↓
[실제 DOM 최소 변경]
```

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

기존 방식 (비효율)

```javascript
<ul>
  <li>A</li>
  <li>B</li>
</ul>
```

→ C 추가 시

```javascript
<ul>
  <li>A</li>
  <li>B</li>
  <li>C</li>
</ul>
```

👉 전체 다시 렌더링 가능성 있음

React 방식

👉 Virtual DOM 비교 결과

```
A (변화 없음)
B (변화 없음)
C (새로 추가)
```

👉 C만 추가

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

3. 선언적 프로그래밍

명령형 프로그래밍 – UI를 어떻게 변경할지(how) 관심

선언적 프로그래밍 – UI를 무엇을 보여줄지(what) 관심

```
const [name, setName] = useState("") //상태 선언

return <h1>{name}</h1>
```

리엑트(React)의 특징

- **리엑트(React)의 핵심 특징**

4. JSX 사용

React는 **JSX(JavaScript XML)** 문법을 사용합니다.

HTML처럼 작성하지만 실제로는 **JavaScript 코드**입니다.

, <hr /> 등 반드시 ('/')로 닫아 주어야 함

```
function App(){
    return<h1>Hello React</h1>;
}
```

리엑트 개발 환경

- **리엑트(React) 개발 환경**

도구	설명
Node.js	JavaScript 실행 환경
npm	패키지 관리
Vite	빠른 React 개발 환경 (CRA 방식은 로딩 속도가 느려서 2025년부터 중지됨)

Node 설치 및 실행

- **Node.js란?**

  Node.js는 구글의 V8 JavaScript 엔진을 기반으로 서버 측에서 JavaScript를 실행할 수 있도록 만든 **오픈소스 런타임 환경**입니다.

  (**웹 브라우저 밖에서도 javascript를 실행**)

- **서버 작업**

  - 웹 서버 만들기
  - API 서버 만들기
  - 파일 처리
  - 데이터베이스 연동

Node 설치 및 실행

- **Node.js 특징**
  - 비동기(Asynchronous) 처리
  - 싱글 스레드 이벤트 루프
  - Npm(Node Package Manager) 패키지 관리자 사용
    (패키지 – 배포할수 있도록 여러 모듈과 모듈 관련 파일을 묶어 놓은 것)

§ 서버 예시 코드

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.write("Hello Node.js");
  res.end();
});

server.listen(3000);
```

백엔드 vs 프런트엔드 프레임워크

- **Express 프레임워크**

  Node.js 기반에서 동작하는 **경량 웹 프레임워크(Web Framework)**입니다.
  주로 **웹 서버와 REST API를 빠르게 구축**하기 위해 사용되며, Node.js 생태계에서 가장 많이 사용되는 서버 프레임워크 중 하나입니다.

- **React 프레임워크**

  React(리액트)는 사용자 인터페이스(UI)를 만들기 위한 **JavaScript 라이브러리** 입니다.
  2013년에 **Meta Platforms (구 Facebook)**에서 공개했으며, 현재 웹 프론트엔드 개발에서 가장 널리 사용되는 기술 중 하나입니다.
  주로 **웹 애플리케이션의 화면(UI)**을 효율적으로 만들기 위해 사용됩니다.

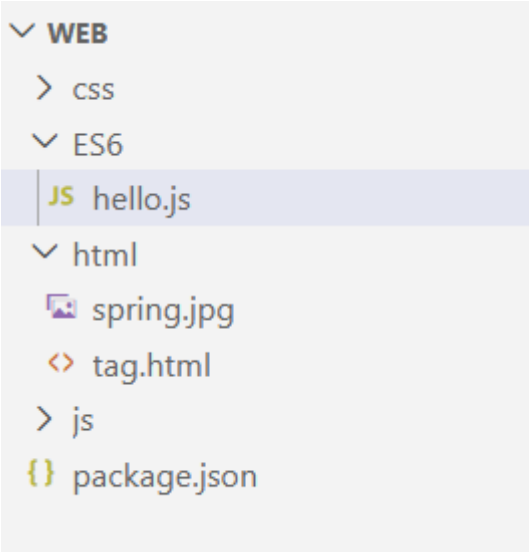
Node 설치 및 실행

- **Node js 다운로드 및 설치**

- **설치 확인 : node -v**

javascript 파일 실행

- **hello.js 만들고 실행하기**




```javascript
// hello 함수 정의
function hello(name) {
    console.log(`Hello, ${name}!`);
}

// hello 함수 호출
hello('World');
```

☞ 터미널에서 실행

₩web₩es6> node hello

javascript ES6

- **ES6(ECMAScript 2015)**

  ES6(ECMAScript 2015)는 자바스크립트의 큰 업데이트로, 코드의 **가독성, 모듈화, 생산성**을 크게 개선한 문법과 기능들이 추가되었습니다.
  현대 프론트엔드 개발(특히 **React**, **Node.js**)에서 기본적으로 사용되는 표준 문법입니다.

- **ES6(ECMAScript 2015) 주요 특징**

  1. let / const (변수 선언)
  2. Template Literal (템플릿 문자열)
  3. Arrow Function (화살표 함수)
  4. Destructuring (구조 분해 할당)
  5. Spread / Rest 연산자 (...)
  6. Module (모듈 시스템) – export/import(이전 – commons, require()사용)

javascript ES6

- **템플릿 문자열 – literal.js**

```
//템플릿 문자열
let season = '봄';

//연결 연산자 - '+'
console.log('현재 계절은 ' + season + '입니다.');

//백틱(`)으로 감싸고, ${}로 변수나 표현식을 감싸면 된다.
console.log(`현재 계절은 ${season}입니다.`);
```

javascript ES6

- **화살표 함수 – function01.js**

```
//함수 선언
let add = function (a, b) {
  return a + b;
}

console.log(add(10, 20));

//화살표 함수
let add2 = (a, b) => {
  return a + b;
}

console.log(add2(10, 20));
```

javascript ES6

- **화살표 함수 – function01.js**

```
//매개변수가 하나인 경우, 괄호 생략 가능
let square = x => {
  return x * x;
}

/*매개변수가 하나이고, 함수 본문이 한 줄인 경우,
  중괄호와 return 생략 가능*/
let square2 = x => x * x;

console.log(square(5));
console.log(square2(5));

//매개 변수 없는 경우
let message = () => console.log('Good, Luck!');
message();
```

javascript ES6

- **구조 분해 할당 – destructure.js**

```
//배열 구조 분해 할당
const arr = [1, 2];
//console.log(arr[0]); //1
//console.log(arr[1]); //2

const [a, b] = arr; //
console.log(`a: ${a}`);
console.log(`b: ${b}`);


//객체 구조 분해 할당
const product = {
  prod_name: '무선마우스',
  price: 27000
};

const { prod_name, price } = product;
console.log(`제품명: ${prod_name}`);
console.log(`가격: ${price}`);
```

javascript ES6

- **Spread / Rest 연산자 (...)**

```
//배열에서 사용
let arr1 = [1, 2, 3];
let arr2 = [4, 5];

//배열을 펼쳐서 새로운 배열을 만든다.
let newArr = [...arr1, ...arr2];
console.log(newArr);

//객체에서 사용
let obj1 = { product: '무선마우스', price: 27000 };
let obj2 = { spec: 'M200 무선마우스 그레이' };

//객체를 펼쳐서 새로운 객체를 만든다.
let combinedObj = { ...obj1, ...obj2 };
console.log(combinedObj);
```

```
[ 1, 2, 3, 4, 5 ]
{ product: '무선마우스', price: 27000, spec: '네이버 M200 무선마우스 그레이' }
```

javascript ES6

- **Array Methods - map() 함수**

  배열의 각 요소에 대해 주어진 함수를 호출한 결과를 모아 **새로운 배열**을 반환하는 함수

```
const arr = [1,2,3];

const newArr = arr.map(x => x * 2);
console.log(newArr); // [2, 4, 6]

const users = [
  { name: 'Jerry', age: 25 },
  { name: 'Linda', age: 30 },
  { name: 'Tom', age: 35 }
];

const names = users.map(user => user.name);
console.log(names); // ['Jerry', 'Linda', 'Tom']

const ages = users.map(user => user.age);
console.log(ages); // [25, 30, 35]
```

javascript ES6

- **ES 모듈 시스템(export / import)**

  package.json에 'type' 속성 추가해야 함

  package.json은 **npm init –y** 명령어로 설치할 수 있음

```
{
  "name": "web",
  "version": "1.0.0",
  "main": "index.js",
  ▷디버그
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "type": "module",
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "ansi-colors": "^4.1.3"
  }
}
```

javascript ES6

- **lib/myAbs.js**

```
function myAbs(x) {
  if (x < 0) {
    return -x;
  } else {
    return x;
  }
}

export { myAbs }; // export { myAbs, square }; 배열로 내보내기
// export default myAbs;
```

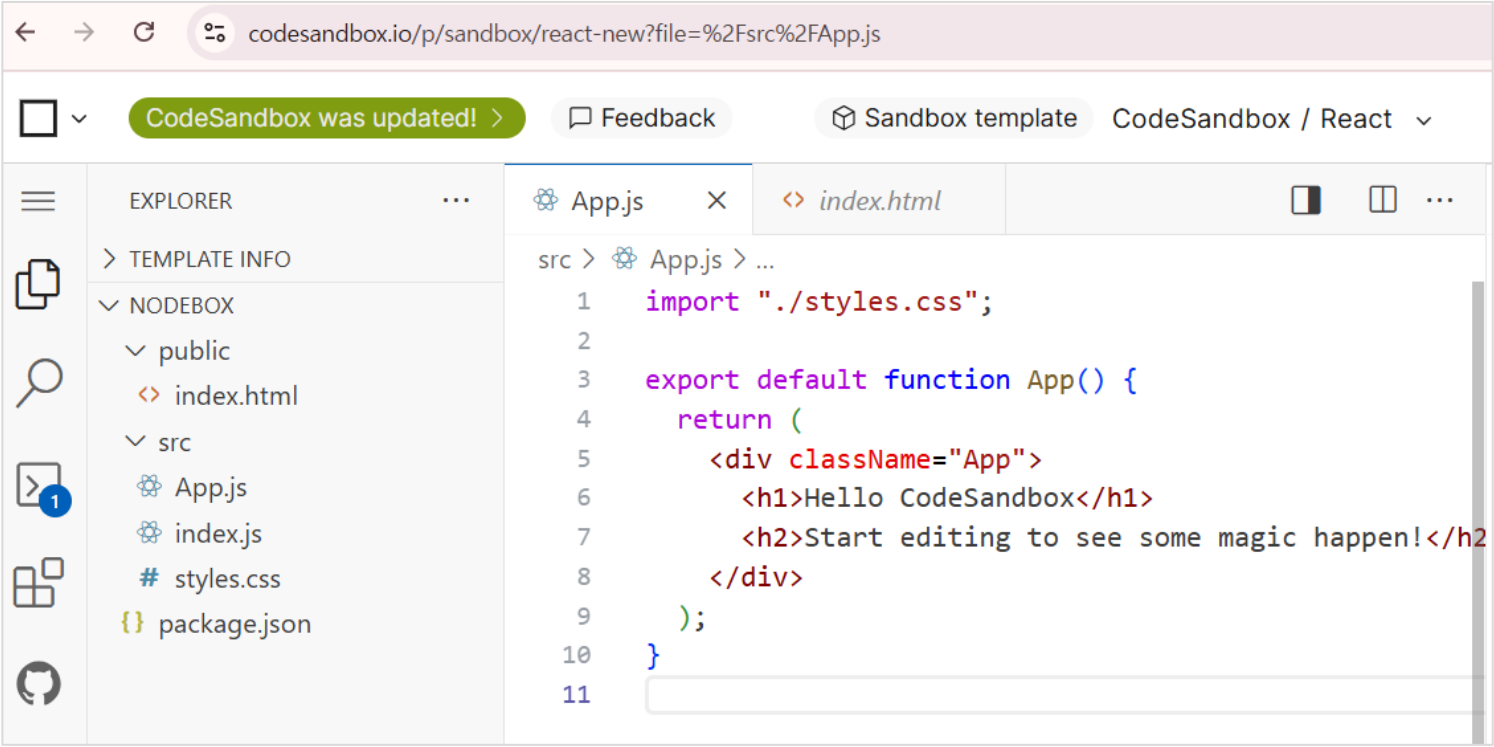
- **main.js**

```
import { myAbs } from "./lib/myAbs.js";

console.log(myAbs(-5));    // 5
console.log(myAbs(10));    // 10
```

리엑트 개발 환경

- **Codesandbox에서 리엑트 사용하기**

signin(로그인)하고 사용



리엑트 개발 환경

- Codesandbox에서 리엑트 사용하기

리엑트 시작하기

Hello~

Hello~ React

```
import "./styles.css";

function Greeting() {
  return <h2>Hello~</h2>;
}

function Greeting2({ name }) {
  return <h2>Hello~ {name}</h2>;
}

export default function App() {
  return (
    <div className="App">
      <h1>리엑트 시작하기</h1>
      <Greeting />
      <Greeting2 name="React" />
    </div>
  );
}
```

Node 설치 및 실행

- **Node js 다운로드 및 설치**

- **설치 확인 : node -v**

Node 설치 및 실행

- **프로젝트 생성 방법**

  1. CRA 방식

  - 명령어: npx create-react-app my-app
  - 서버 시작 & 반영 속도 느림(점점 사용 감소)

  2. Vite 빌드 도구 사용 방식

  - 명령어: npm create vite@latest my-app
  - 서버 시작 & 반영 속도 매우 빠름(대중적)

리엑트 애플리케이션 만들기

- **Vite로 프로젝트 생성하기**

c:/reactworks> npm create vite@latest my-app

```
◆ Select a framework:
│ ○ Vanilla
│ ○ Vue
│ ● React
│ ○ Preact
│ ○ Lit
```

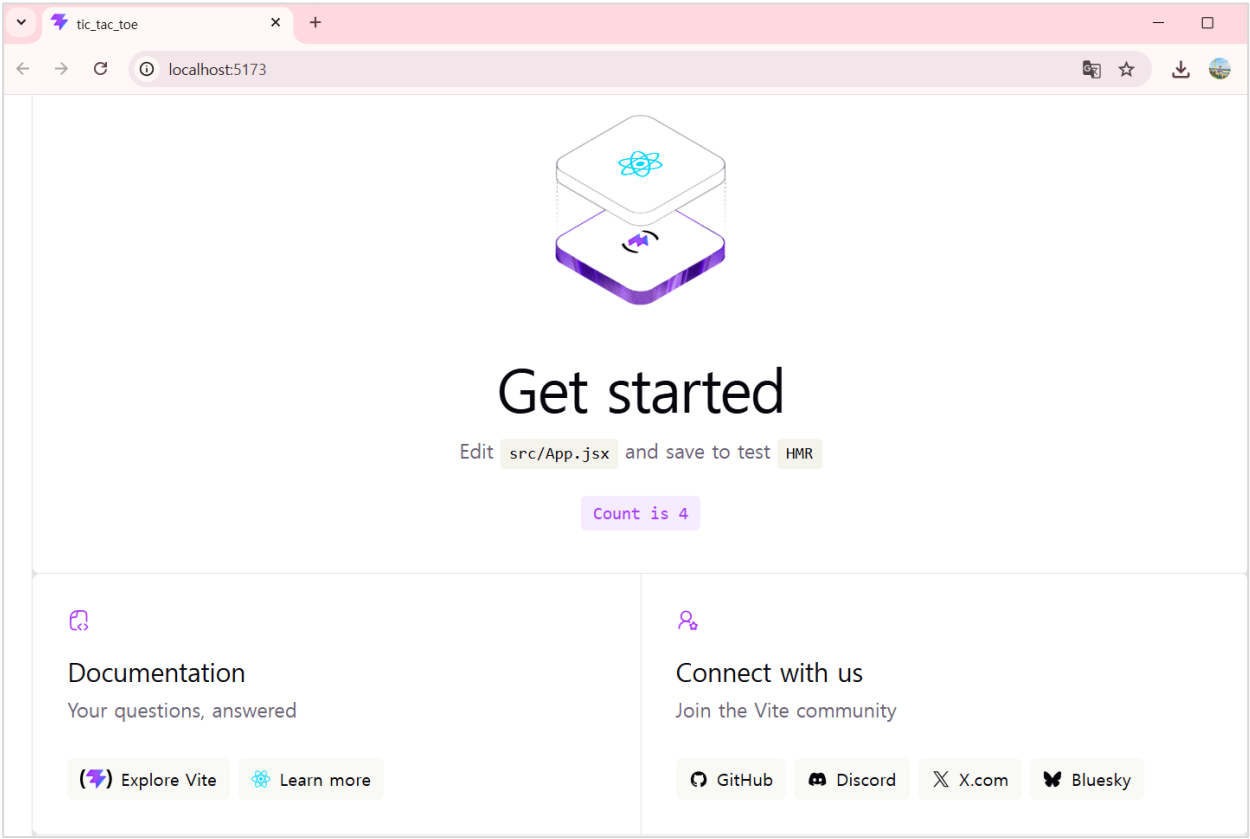
```
Select a variant:
○ TypeScript
○ TypeScript + React Compiler
○ TypeScript + SWC
● JavaScript
```

```
VITE v7.3.1  ready in 561 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

- 서버 중지 – ctrl + C
- 실행 명령 : npm run dev

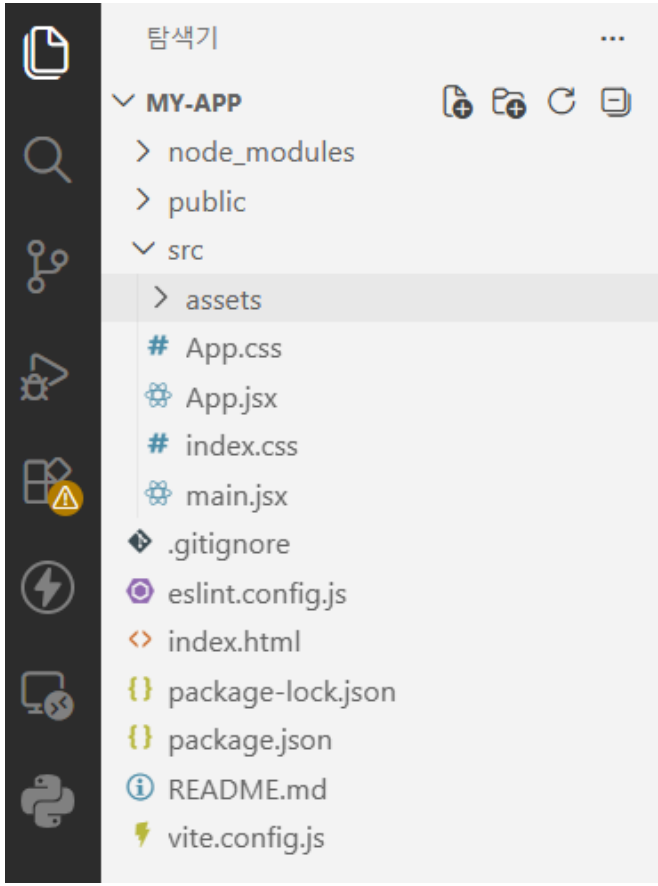
리엑트 애플리케이션 만들기

- **Vite로 프로젝트 생성하기**



리엑트 애플리케이션 만들기

- **프로젝트 구조**



리엑트 애플리케이션 만들기

- **index.html**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial
    <title>my-app</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

리엑트 애플리케이션 만들기

- **main.jsx**

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

리엑트 애플리케이션 만들기

- **App.jsx**

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
```

리엑트 애플리케이션 만들기

- **App.css**

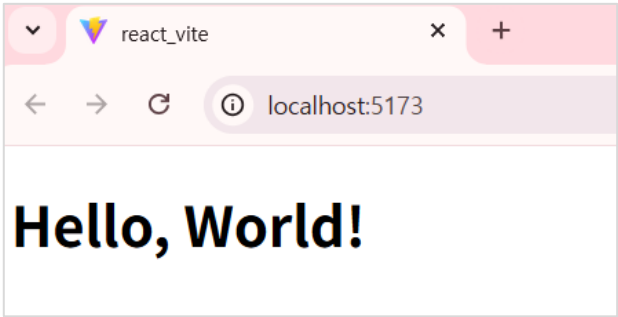
```
@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}
```

리엑트 애플리케이션 만들기

- **App.jsx 변경하기**

  리엑트 파일 중 첫 글자가 대문자로 시작하고 확장자가 jsx인 파일을 컴포넌트(component) 라고 합니다.
  App.jsx 기존 코드는 모두 지우고, 출력 코드 작성. (App.css, index.css 코드도 모두 삭제)

```
import './App.css'

function App() {

  return (
    <div>
      <h1>Hello, World!</h1>
    </div>
  )
}

export default App
```



리엑트 애플리케이션 만들기

- **package.json**

  프로젝트의 핵심 설정 파일입니다.

  dependencies는 설치된 라이브러리 정보로 의존성 관리 키(key)입니다.

```
{
  "name": "react_vite",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  ▷디버그
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  },
```

리엑트 애플리케이션 만들기

- **전체 실행 과정**

1) npm run dev로 서버를 실행합니다.

2) 서버는 index.html 파일을 엽니다.

3) index.html 파일은 main.jsx 파일을 불러 옵니다.

4) main.jsx 파일에서 App 컴포넌트를 읽어 실행합니다.

5) App 컴포넌트가 웹 브라우저 화면에 표시됩니다.

JSX 알아보기

- **JSX(JavaScript XML) 란?**

  React에서 사용하는 **JSX**는 HTML과 매우 비슷하게 생겼지만 실제로는 **JavaScript 문법 확장**입니다.

  그래서 개발할 때 몇 가지 중요한 차이점이 있습니다.

- **JSX vs HTML**

구분	HTML	JSX
클래스	class	className
태그 구조	여러 태그 가능	하나의 부모 요소 필요
태그 닫기	일부 생략 가능	반드시 닫아야 함
JS 사용	불가능	`{}` 안에서 가능
이벤트	onclick	onClick

JSX 알아보기

- **JSX 문법**
  - ✓ 2개의 태그를 병렬로 사용할 수 없어서 <><> Fragment 태그를 추가함
  - ✓
 <hr />등의 태그는 '/(슬래시)'로 꼭 닫아야 함.

```
return (
  <>
    <div>
      <h1>Hello, World!</h1>
    </div>

    <div>
      <h2>welcome!<br />홈페이지 방문을 환영합니다.</h2>
    </div>
  </>
)
```

<오류!>

JSX 알아보기

- **JSX 문법**

  ✓ 선택자는 className을 사용함(class 사용하면 오류 발생!)

App.jsx

```jsx
import './App.css' // CSS 파일을 import하여 스타일을 적용합니다.

function App() {

  return (
    <>
      <div>
        <h1>Hello, World!</h1>
      </div>

      <div className="welcome">
        <h2>Welcome~ <br /> 홈페이지 방문을 환영합니다!</h2>
      </div>
    </>
  )
}
```

App.css

```css
.welcome h2{
  color: #00f;
  font-weight: bold;
}
```

JSX 알아보기

- **JSX 문법**
  - ✓ style 속성에 문자열이 아닌 자바스크립트 객체를 할당함.
    (키는 카멜케이스 형태로 작성함)
  - ✓ 주석은 중괄호 안에 /* ~ */ 형태로 작성함

```
return (
  <>
    <div>
      {/* 인라인 스타일 적용 */}
      <h1
        style={{
          color: 'red',
          fontWeight: 'bold'
        }}
      >Hello, World!</h1>
    </div>
```

JSX 알아보기

- JSX 문법

현재 계절은 봄입니다.

JSX 알아보기

- **JSX 문법**

  ✓ 변수 또는 표현식은 **중괄호 { }** 안에서 사용함

```
import flower from './assets/flower.jpg'

function App() {
  const season = "봄";

  return (
    <>
      <h2>현재 계절은 {season}입니다.</h2>
      <img
        src={flower}
        alt="꽃"
        width="400"
      />
```

JSX 실습 문제

- **JSX 실습 문제**

김기용

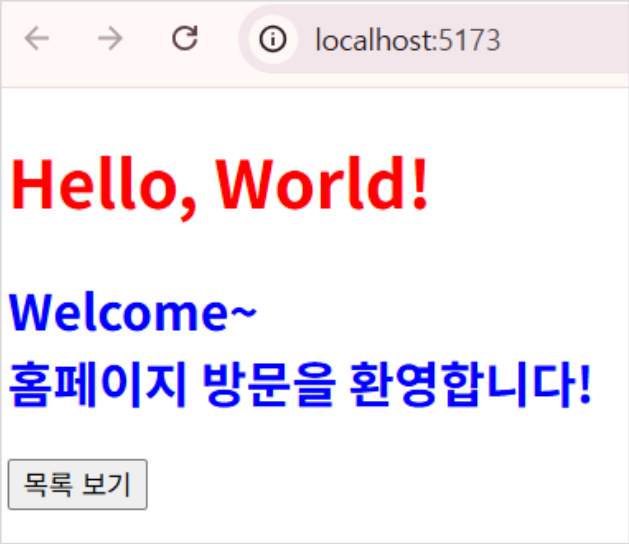
```
import './App.css' // CSS 파일을 import하여 스타일
import profilePhoto from './assets/my-photo.jpg'
```

```
function App() {
  const season = "봄";

  // 사용자 정보 객체 정의
  const user = {
    name: '김기용',
    imageUrl: profilePhoto,
  }

  return (
    <>
      <section>
        <h1>{user.name}</h1>
        <img
          className="profile-photo"
          src={user.imageUrl}
          alt={'Photo of ' + user.name}
        />
      </section>
```

```
body{
  background-color: #f0f0f0;
  margin: 0;
  padding: 20px;
}

.profile-photo{
  width: 160px;
  height: 160px;
  border-radius: 50%;
}
```

컴포넌트(Component)

- 컴포넌트 사용하기




```
// 버튼 컴포넌트 정의
function MyButton() {
  return (
    <button>
      목록 보기
    </button>
  );
}
```

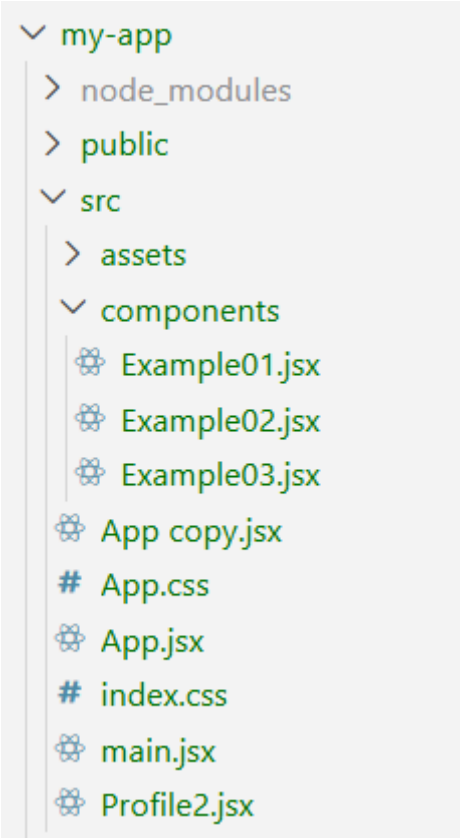
컴포넌트(Component)

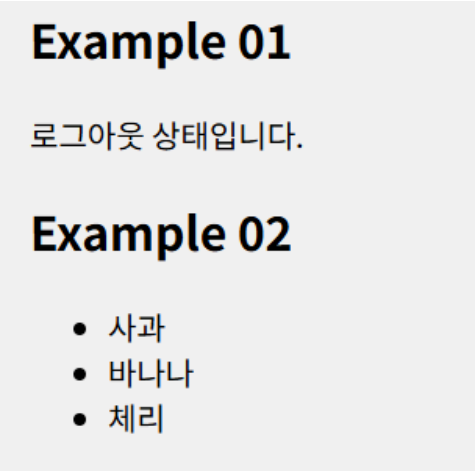
- 컴포넌트 사용하기

```
        <div className="welcome">
          <h2>Welcome~ <br /> 홈페이지 방문을 환영합니다!</h2>
        </div>
        {/* 버튼 컴포넌트 사용 */}
        <MyButton />
      </>
    )
}
```

외부 파일 컴포넌트

- **components 폴더 생성 > Example01.jsx 파일 생성**





조건부 랜더링

- **조건부 랜더링 – 조건문 구현**

```
const Example01 = () => {
  // 조건부 랜더링
  const isLoggedIn = false;

  return (
    <div>
      <h2>Example 01</h2>
      {/* 조건부 랜더링 - 삼항 연산자 */}
      {isLoggedIn ? <p>로그인 상태입니다.</p> : <p>로그아웃 상태입니다.</p>}

      {/* && 연산자를 사용한 조건부 랜더링 */}
      {isLoggedIn && <p>로그인 상태입니다.</p>}
    </div>
  )
}

export default Example01
```

리스트 랜더링

- **리스트 랜더링 – 반복문 구현**

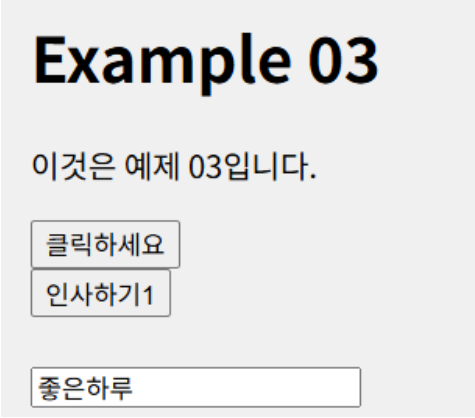
```
const Example02 = () => {
  // 리스트 랜더링
  const items = ['사과', '바나나', '체리'];

  return (
    <div>
      <h2>Example 02</h2>
      {/* 리스트 랜더링 */}
      <ul>
        {items.map((item, index) => (
          // key prop을 사용하여 각 항목에 고유한 키를 부여합니다.
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  )
}

export default Example02
```

key 없으면 브라우저 콘솔에 경고 발생

```
▶ Each child in a list should have a unique "key" prop.

Check the render method of `Example02`. See https://react.dev/link/warning-k
information.
```

이벤트 핸들링

- **이벤트 핸들러**
  - 이벤트 핸들러는 이벤트가 발생했을때 실행할 함수입니다.
    (함수 참조 – 컴포넌트 밖에 따로 정의하는 방식)
  - 인라인 핸들러 – 이벤트 속성안에 함수를 직접 생성하는 방식



이벤트 핸들링

- 이벤트 핸들러

```
const Example03 = () => {
  // 이벤트 핸들링
  const handleClick = () => {
    alert('버튼이 클릭되었습니다!');
  }

  const greet = (name) => {
    alert(`안녕하세요, ${name}님!`);
  }

  // 폼 요소와 이벤트 핸들링
  const handleInputChange = (event) => {
    console.log(event); // 이벤트 객체 전체 출력
    // 입력된 값 출력
    console.log(event.target.value);
  }
```

이벤트 핸들링

- 이벤트 핸들러

```
return (
  <div>
    <h1>Example 03</h1>
    <p>이것은 예제 03입니다.</p>
    {/* 이벤트 핸들링 - 클릭 이벤트 */}
    <button onClick={handleClick}>클릭하세요</button>
    <br />
    {/* 화살표 함수를 사용한 이벤트 핸들링 */}
    <button onClick={() => greet('김도영')}>인사하기1</button>
    <br />
    <br />
    {/* 폼 요소와 이벤트 핸들링 - 입력 이벤트 */}
    <input type="text" onChange={handleInputChange} />
  </div>
)
```

```
▸ nativeEvent: InputEvent {isTrusted: true,
▾ target: input
    value: "apple"
```

이벤트 핸들링

- 이벤트 핸들러 – 매개변수가 있는 함수

```
// 절댓값을 구하는 함수
const handleMyAbs = (x) => {
  console.log(Math.abs(x)) // Math.abs() 함수를 사용
  if (x < 0) {
    console.log(-x)
  } else {
    console.log(x)
  }
}
```

```
<button
  onClick={
    () => handleMyAbs(-5)
  }>절댓값
</button>
```

실습 문제

- **문제**

  사용자 이름을 변수로 선언하고 JSX에서 출력하세요

  예)

  ```
  이선화님 환영합니다.
  ```

실습 문제

- **문제**

  조건부 렌더링을 사용하여 로그인 상태를 표시하세요.

  조건

  ```
  로그인 -> 환영합니다.
  로그아웃 ->로그인 해주세요
  ```

실습 문제

- **문제**

  컴포넌트를 3개로 분리하세요.

```
Header
Main
Footer
```

실습 문제

- **문제**

  버튼을 클릭하면 console.log("클릭됨")이 출력되도록 만드세요.